Week 12: Catalan bijections

Adult guide built from the problem files · Unpiloted. Physical pretests unperformed.

Each problem's key repeats the main answer from the print guide, week-12-facilitator.pdf, whose source is editable/facilitator-src/build_guide.py; the held hints, galleries and proofs are on the guide page named in the key's first line. The overview repeats the guide's first page. Change both together; check_guide_copy.py checks that every sentence here is in the guide.

Mathematical overview

The mathematical facts

Three equivalent objects. For each nonnegative integer n, these are in bijection: complete noncrossing pairings of 2n fixed points around a circle, with strings inside it (or in a row, with arcs above it); paths of n up-right steps U and n down-right steps D that start and end at height zero and never go below it; and finite ordered rooted trees with n edges. A tree is connected and has no cycles; the children at every vertex have a fixed left-to-right order. Circle labels stay fixed: rotations or reflections do not identify different partner lists.

The maps are reversible. Read pairing endpoints in order, writing U at a pair's first end and D at its other end. At each D, the inverse closes the most recent unclosed U; the noncrossing rule forces this choice. For a tree, visit whole child branches left-to-right, writing U when moving away from the root and D when returning. Its inverse adds a new rightmost child at U and moves to the parent at D. These rules prove the bijections; equal counts alone do not.

The counts. Let C(n) count any one of these sets and let C(0)=1. For n at least 1, C(n) is the sum of C(i)C(n-1-i) for i from 0 to n-1. The unique first-return split U P D Q proves this recurrence. Thus C(0),...,C(6) = 1, 1, 2, 5, 14, 42, 132. The full split argument is on page 9.

Parity and neighbors. An odd number of points cannot be completely paired. On a circle, a fixed chord can be completed without crossings exactly when it leaves an even number of other points on each side. Every nonempty complete noncrossing circle pairing contains a pair of circular neighbors. The proofs and exact instances follow on pages 3-4.

Where the worksheets investigate these facts

K-1 P1-P2: Find the five six-dot pairings, then justify completeness. The two four-dot cases are printed examples.

K-1 P3-P6: Complete fixed chords; explore eight-dot pairings; test parity; decide whether neighbors can be avoided.

Grades 2-3 P1-P4: Enumerate six-dot pairings; encode paths; investigate the inverse; match all five paths and pairings.

Grades 2-3 P5-P6: Decide which codes are legal; enumerate all 14 eight-dot pairings.

Grades 4-5 P1-P4: Enumerate three-edge trees; encode and reconstruct trees; investigate uniqueness and characterize legal codes.

Grades 4-5 P5-P6: Count trees with four, five and six edges; propose and justify a general counting rule.

The examples introduce a representation or a walk, not a counting method. Let children conjecture before giving the facts above. Use drawings and explanations; keep hints and formal recurrence notation until needed.

Problems and routes

Problem	K–1	2–3	4–5	route
Every pairing of six dots on a circle	P1			first
Show that no six-dot pairing is missing	P2			first
Finish an eight-dot pairing from a drawn pair	P3			first/return
Many pairings of eight dots	P4			return
Which circles can be completely paired?	P5			first
Pair eight dots without joining neighbors?	P6			return
Every pairing of six dots in a row		P1		first
Paths for three six-dot pairings		P2		first
Pairings for four paths, and whether a path has two		P3		first/return
Every six-step path, matched to the pairings		P4		first/return
Which letter strings are pairing codes?		P5		first/return
Every pairing of eight dots in a row		P6		return
Every ordered tree with three edges			P1	first
Walk codes for four trees			P2	first
Trees for four codes, and whether a code has two			P3	first/return
Rules for which codes come from trees			P4	first/return
How many trees have four edges?			P5	return
Five and six edges, and a counting rule			P6	return

First route. K–1 P1–P3, P5; Grades 2–3 P1–P5; Grades 4–5 P1–P4. **Return visits.** K–1 P3–P4, P6; Grades 2–3 P3–P6; Grades 4–5 P3–P6.

Page numbers. A key that names a student page counts the pages of the full band packet. The route packets keep the problem numbers but number their own pages; each route-packet page is this page of the full packet:

Route packet	its pages = full-packet pages
K–1 first	pp. 1–3 = pp. 1–3; p. 4 = p. 6
K–1 return	p. 1 = p. 1; pp. 2–4 = pp. 3–5; p. 5 = p. 7
Grades 2–3 first	pp. 1–5 = pp. 1–5
Grades 2–3 return	pp. 1–6 = pp. 1–6
Grades 4–5 first	pp. 1–4 = pp. 1–4
Grades 4–5 return	pp. 1–6 = pp. 1–6

Key, one entry per problem

Every pairing of six dots on a circle

K–1 P1 · first route

Idea. Six fixed dots on a circle have exactly five noncrossing pairings: dot 1 can meet only 2, 4 or 6, which leave 2, 1 and 2 ways to finish.

Print guide, p. 3. The printed four-dot examples are 12|34 and 14|23; 13|24 crosses. The task is to find all five six-dot pairings in the gallery on page 6. Here 12 means the pair {1,2}; a vertical bar separates pairs. Do not count a rotated drawing as the same unless every numbered dot still has the same partner.

Show that no six-dot pairing is missing

K–1 P2 · first route

Idea. Sorting by dot 1's partner (2, 4 or 6) proves the five pairings complete: a string must leave an even number of dots on each side.

Print guide, p. 3. Check the P1 collection for repeats or omissions. Exactly five pairings: 12|34|56, 12|36|45, 14|23|56, 16|23|45, 16|25|34. To establish completeness, 1 can meet only 2, 4 or 6; these give 2, 1 and 2 completions. A chord leaves an even number of dots on each side.

Finish an eight-dot pairing from a drawn pair

K–1 P3 · first/return route

Idea. A fixed string can be completed without crossings exactly when it leaves an even number of dots on each side: 1-4 and 1-6 finish two ways each, 1-3 and 1-5 trap an odd group.

Print guide, p. 4. Keep each supplied chord. For 14, completions are 14|23|56|78 and 14|23|58|67. For 16, they are 16|23|45|78 and 16|25|34|78. The 13 and 15 starts are impossible: they trap respectively one or three dots on one side, and a string cannot leave without crossing the fixed chord. Every start is printed twice, so the frames do not show which starts can be finished: the 14 and 16 frames hold the two completions of each, and the 13 and 15 frames hold none.

Many pairings of eight dots

K–1 P4 · return route

Idea. Eight fixed dots have 14 noncrossing pairings; dot 1's partner 2, 4, 6 or 8 leaves 5, 2, 2 and 5 ways to finish.

Print guide, p. 4. There are 14 possible eight-dot pairings, listed and drawn on page 7. The task asks for as many as possible, so a well-organized partial collection is a valid stopping point. Group by 1's partner: 2, 4, 6, 8 give 5, 2, 2, 5 completions. Pages 4 and 5 hold 16 frames for the 14 pairings, so the frames do not give away the count. Two drawings are the same only if every dot has the same partner. A match at one dot is not a repeat: 12|34|56|78 and 12|38|45|67 agree at dot 1 but are different.

Which circles can be completely paired?

K–1 P5 · first route

Idea. An even number of dots can always be paired (join neighbors); an odd number never can, since each string removes two dots.

Print guide, p. 4. Ten, four and six dots are possible: join adjacent pairs (12,34,56,78,9-10 as far as the circle allows). Three, five and seven dots are impossible. Each string removes two unpaired dots, so an odd starting number always leaves an odd number; it cannot reach zero. A physical demonstration that repeatedly removes pairs is sufficient.

Pair eight dots without joining neighbors?

K–1 P6 · return route

Idea. Every noncrossing pairing joins some two neighbors: an innermost string has no dots under it.

Print guide, p. 4. No. Every noncrossing complete pairing has a pair of circular neighbors. Proof: consider any chord and one side of it. If there are dots on that side, they must be paired entirely within that side. Choose a pair with the fewest intervening dots in the linear reading order 1,...,8. If it had any, those dots would have a pair between them with fewer intervening dots, a contradiction. Thus some consecutive numbered pair is joined, and it is also a pair of circular neighbors.

Every pairing of six dots in a row

Grades 2–3 P1 · first route

Idea. Six fixed dots in a row have exactly five noncrossing pairings, split 2 + 1 + 2 by dot 1's partner.

Print guide, p. 5. The ten-dot example is 14|23|5-10|67|89. The six-dot task has the same five pairings as K-1 P1-P2; use the page 6 gallery. Above a line, the endpoints still have fixed order. Completeness is the same 2+1+2 split by point 1's partner.

Paths for three six-dot pairings

Grades 2–3 P2 · first route

Idea. Reading a noncrossing pairing as U at each first end and D at each second end gives a path that never dips below its start: its height counts the strings still open.

Print guide, p. 5. Worked example: UDUUDDUD, from 12|36|45|78. For the three unanswered cases, top to bottom: UUUDDD, UUDDUD, UDUDUD. Their height sequences, including the start, are 0,1,2,3,2,1,0; 0,1,2,1,0,1,0; and 0,1,0,1,0,1,0. All stay at or above the starting level and end there. Height counts opened pairs that have not yet closed; that number cannot be negative.

Pairings for four paths, and whether a path has two

Grades 2–3 P3 · first/return route

Idea. Each path has exactly one noncrossing pairing: at every D, close the most recent open U (the stack rule), and closing an older one would force a crossing.

Print guide, p. 5. Top to bottom: UUDUDD gives 16|23|45; UDUUDD gives 12|36|45; UUDDUUDD gives 14|23|58|67; UUUDUDDD gives 18|27|34|56. No path has two noncrossing inverse pairings. At every D, close the most recently opened unclosed U. If an older opening were closed first, the more recent opening would have to close later, causing interleaving endpoints and a crossing. This is the stack rule, and it is forced, not merely a convenient algorithm.

Every six-step path, matched to the pairings

Grades 2–3 P4 · first/return route

Idea. There are exactly five such paths, and the code and the stack rule undo each other, so paths and pairings match one to one.

Print guide, p. 5. Exactly five paths; see page 6. Match them to the pairings using the unique inverse just proved. A count of five in both lists alone is not a proof of a bijection: the rules in both directions must undo one another.

Which letter strings are pairing codes?

Grades 2–3 P5 · first/return route

Idea. A U/D string is a pairing code exactly when its height never goes below zero and ends at zero; the two failures are a D with no open U and U's left open.

Print guide, p. 5. UUUDDUDD is valid: 18|25|34|67. UDDUUDUD is invalid: after its third letter the height is -1, so a closing endpoint has no opening partner. UUDUDUDD is valid: 18|23|45|67. UUUUDDUD is invalid: five U and three D leave height 2 and two unmatched openings.

Every pairing of eight dots in a row

Grades 2–3 P6 · return route

Idea. Eight dots have 14 noncrossing pairings: dot 1's partner splits them 5 + 2 + 2 + 5.

Print guide, p. 5. Exactly 14 pairings, page 7. The four cases for the partner of 1 give 5+2+2+5. Within each case, choices inside and after the first pair are independent. The forced-split proof on page 9 justifies completeness.

Every ordered tree with three edges

Grades 4–5 P1 · first route

Idea. There are exactly five ordered rooted trees with three edges; sorting by the number of root children gives 2 + 2 + 1.

Print guide, p. 8. The five-edge example has two root children: the left has two leaf children, and the right has one. Exactly five three-edge trees; see page 6 and its root-child case split. A chain and a fork are different; changing edge lengths is irrelevant. Swapping unequal subtrees creates a different ordered tree.

Walk codes for four trees

Grades 4–5 P2 · first route

Idea. A tree's walk code shows its edges (half the length), depth (greatest height), leaves (UD peaks) and root children (returns to zero).

Print guide, p. 8. The worked two-edge fork has code UDUD: visit the left leaf and return, then visit the right leaf and return. U is away from the root on the tree and upward on the path. For the four unanswered trees, reading top-left, top-right, bottom-left, bottom-right: UUUUDDDD, UDUDUDUD, UUDUDDUD, UDUUUDDD. Number of edges is half the code length. Maximum height is maximum root-to-vertex distance. A UD peak records a leaf. Every return to zero finishes a root child's branch, so the number of such returns is the root's number of children.

Trees for four codes, and whether a code has two

Grades 4–5 P3 · first/return route

Idea. Each legal code builds exactly one ordered tree: at U add a new rightmost child and move to it, at D move to the parent.

Print guide, p. 8. No. For these ordered rooted trees, each legal complete code describes exactly one tree. UUDDUD: the root has two children; the left child has one child. UDUUDD: the root has two children; the right child has one child. UUDUDUDD: the root has one child, and that child has three leaf children. UUUDDUDD: the root has one child; that child has two children, the left of which has one child. At U there is one prescribed action: add a new rightmost child of the current vertex and move there. At D move to the parent. No choices remain, so there is exactly one ordered tree.

Rules for which codes come from trees

Grades 4–5 P4 · first/return route

Idea. A U/D string comes from a tree exactly when every prefix has at least as many U's as D's and the totals are equal; the inverse construction then rebuilds the tree.

Print guide, p. 8. Valid: UUUUDDDD, UDUDUUDD, UUUDUDDD. Invalid: UUDDDUUD first goes below zero at letter 5; DUUUDDUD does so at letter 1; UUDUDUDU finishes at height 2. Necessary and sufficient rules: every prefix has at least as many U as D, and total U equals total D. The empty code gives the root-only tree. The construction from P3 never asks for a parent above the root and finishes at the root. Walking its new children left-to-right gives back the code, which proves sufficiency and reversibility.

How many trees have four edges?

Grades 4–5 P5 · return route

Idea. There are 14 four-edge trees; splitting each code at its first return to zero counts every tree exactly once.

Print guide, p. 9. There are 14 four-edge trees. The page 7 gallery exhibits them; this argument certifies none are missed. A nonempty legal code has exactly one split U P D Q, with the displayed D being the first return to height zero. P and Q are themselves legal codes. If P has i pairs, Q has n-1-i pairs. Choosing them independently gives C(i) times C(n-1-i) choices, and different i or different smaller words cannot give the same whole word.

Five and six edges, and a counting rule

Grades 4–5 P6 · return route

Idea. With C(0) = 1, C(n) is the sum of C(i)C(n-1-i) for i from 0 to n-1, giving 1, 1, 2, 5, 14, 42, 132; the first-return split proves it.

Print guide, p. 9. Let C(0)=1, the one empty code or root-only tree. For n at least 1, C(n) is the sum of C(i)C(n-1-i) for i=0,...,n-1. The values C(0) through C(6) are 1,1,2,5,14,42,132. In particular C(4)=5+2+2+5=14; C(5)=14+5+4+5+14=42; C(6)=42+14+10+10+14+42=132. This is a disjoint, exhaustive decomposition, not extrapolation from a number pattern.

In a tree, remove the edge from the root to its first child. P describes that child's subtree; Q describes the remaining root branches. Conversely attach any P tree as the new leftmost child branch of any Q tree. These operations undo one another.